

بسمه تعالی

درس سیستم های نهفته - نیمسال تحصیلی ۰۴-۰۵

دانشگاه صنعتی شریف - دانشکده مهندسی برق



گزارش بخش چهارم

امیرحسین مطیعی - شماره دانشجویی ۴۰۱۱۰۲۵۵۳

گزارش بخش چهارم: خواندن اطلاعات سنسورها با پروتکل SNMP

مقدمه

در بخش چهارم پروژه، هدف اصلی فراهم کردن امکان خواندن اطلاعات سنسورهای سیستم توزیع شده با استفاده از پروتکل SNMP بود. سیستم پیاده سازی شده شامل یک نود Master و دو نود Slave است و هر نود پایگاه داده SQLite مستقل خود را دارد. در بخش های قبلی، اطلاعات سنسورها و مقادیر ثبت شده آنها در این پایگاه های داده ذخیره شده بود. در این بخش، یک Agent مبتنی بر snmpd روی هر نود راه اندازی شد تا اطلاعات سنسورها از طریق OID های اختصاصی قابل دسترسی باشد. برای اتصال snmpd به پایگاه داده، از قابلیت pass در Net-SNMP استفاده شد. این قابلیت درخواست های مربوط به یک شاخه مشخص از OID ها را به یک برنامه خارجی منتقل می کند. برنامه خارجی این پروژه با زبان ++C نوشته شده و وظیفه آن دریافت درخواست SNMP، خواندن اطلاعات لازم از SQLite و تولید پاسخ در قالب مورد انتظار Net-SNMP است. در نتیجه، هر سه نود به صورت مستقل اطلاعات سنسورهای محلی خود را ارائه می کنند و نود Master می تواند از طریق شبکه اطلاعات هر دو Slave را نیز بخواند.

آشنایی با پروتکل SNMP

SNMP مخفف عبارت Simple Network Management Protocol است و یکی از پروتکل های متداول برای مانیتورینگ، مدیریت و جمع آوری اطلاعات از تجهیزات و سرویس های شبکه به شمار می رود. با استفاده از SNMP می توان اطلاعاتی مانند وضعیت سیستم، مصرف منابع، وضعیت رابط های شبکه، اطلاعات تجهیزات و مقادیر سنسورها را از راه دور دریافت کرد.

معماری SNMP معمولاً از دو بخش اصلی تشکیل می شود. بخش نخست SNMP Manager است که درخواست های مدیریتی یا مانیتورینگ را ارسال می کند. بخش دوم SNMP Agent است که روی سیستم تحت نظارت اجرا می شود و اطلاعات مورد درخواست را در اختیار Manager قرار می دهد.

در این پروژه، ابزارهایی مانند snmpget و snmpwalk نقش Manager را دارند و سرویس snmpd روی هر یک از سه ماشین مجازی نقش Agent را ایفا می کند. درخواست ها از طریق شبکه و با استفاده از UDP ارسال می شوند. برای جلوگیری از تداخل با سرویس پیش فرض SNMP سیستم که معمولاً روی پورت ۱۶۱ اجرا می شود، Agent سفارشی پروژه روی پورت UDP شماره ۱۱۶۱ راه اندازی شد. نسخه مورد استفاده در این پیاده سازی Net-SNMP 5.9.1 است و تمام نودها از سیستم عامل Ubuntu 22.04 استفاده می کنند.

```

===== NODE AND OPERATING SYSTEM =====
Hostname: embedded-master
IP addresses: 192.168.122.22
Operating system: Ubuntu 22.04.5 LTS

===== REQUIRED PACKAGES =====
build-essential 12.9ubuntu3
libsqlite3-dev 3.37.2-2ubuntu0.7
snmp 5.9.1+dfsg-1ubuntu2.9
snmpd 5.9.1+dfsg-1ubuntu2.9

===== TOOL VERSIONS =====

NET-SNMP version: 5.9.1
g++ (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
SQLite version: 3.37.2

```

مفهوم OID در SNMP

در SNMP، هر داده قابل خواندن یا قابل مدیریت با یک شناسه سلسله مراتبی به نام OID مشخص می شود. Object Identifier. این شناسه از مجموعه ای از اعداد تشکیل شده که با نقطه از یکدیگر جدا شده اند. ساختار درختی OID باعث می شود هر متغیر در فضای مدیریتی SNMP دارای یک آدرس منحصر به فرد باشد.

برای نمونه، OID اصلی تعریف شده در این پروژه به شکل زیر است:

1.3.6.1.4.1.8072.9999.4

بخش 1.3.6.1.4.1 مربوط به شاخه خصوصی شرکت ها و سازمان ها در ساختار استاندارد SNMP است. عدد 8072 شناسه ای است که در فضای Net-SNMP استفاده می شود و بخش 9999.4 برای شاخه اختصاصی سنسورهای بخش چهارم پروژه در نظر گرفته شده است. تمام اطلاعات سنسورها در زیرمجموعه این OID قرار دارند. استفاده از یک OID پایه ثابت و قرار دادن متغیرهای مختلف در شاخه های زیرین آن باعث شده ساختار اطلاعات منظم، قابل گسترش و قابل پیمایش با snmpwalk باشد.

مفهوم MIB

MIB مخفف Management Information Base است و ساختار منطقی اطلاعات قابل دسترسی از طریق SNMP را تعریف می کند. فایل MIB مشخص می کند هر OID چه مفهومی دارد، نوع داده آن چیست و در چه بخشی از درخت SNMP قرار گرفته است. در این پروژه فایل زیر برای مستندسازی ساختار MIB سفارشی سنسورها در نظر گرفته شده است:

snmp/HOTEL-SENSOR-MIB.txt

وجود فایل MIB باعث می شود ساختار OID ها تنها مجموعه ای از اعداد نباشد و بتوان برای هر بخش نام و مفهوم مشخصی تعریف کرد. با این حال، برای آنکه خروجی تست ها مستقل از تنظیمات MIB سیستم باشد، بیشتر آزمایش ها با گزینه -On انجام شدند. این گزینه باعث می شود OID ها به شکل عددی کامل نمایش داده شوند.

ساختار فایل های بخش چهارم

ساختار بخش چهارم به گونه ای طراحی شد که تنظیمات، کد اصلی، اسکریپت ها و خروجی های تولید شده از یکدیگر جدا باشند. پوشه `config` شامل تنظیمات سه نود و فهرست آدرس های شبکه است. پوشه `snmp` شامل کد ++C ، `Makefile`، فایل `MIB` و قالب پیکربندی `snmpd` است. پوشه `scripts` نیز اسکریپت های نصب، کامپایل، اجرا، توقف و تست را در خود جای داده است.

پوشه `generated` در هنگام اجرا برای نگهداری فایل های پیکربندی اختصاصی هر نود، فایل `PID` ، `Log` و اطلاعات پایدار-`Net-SNMP` استفاده می شود. پوشه `test-output` نیز برای ذخیره خروجی تست های `snmpwalk` و سایر آزمایش ها در نظر گرفته شده است.

```
├── config
│   ├── master.env
│   ├── nodes.env
│   ├── nodes.env.example
│   ├── slave1.env
│   └── slave2.env
├── generated
├── Makefile
├── README.md
├── scripts
│   ├── build_and_run.sh
│   ├── common.sh
│   ├── generate_snmpd_config.sh
│   ├── install_dependencies.sh
│   ├── read_all_sensors.sh
│   ├── start_snmpd.sh
│   ├── stop_snmpd.sh
│   ├── test_pass_handler.sh
│   └── test_snmp.sh
├── snmp
│   ├── HOTEL-SENSOR-MIB.txt
│   ├── Makefile
│   ├── sensor_snmp_pass.cpp
│   └── snmpd.conf.template
└── test-output

6 directories, 20 files
```

نصب بسته‌های موردنیاز

برای اجرای بخش SNMP، ابتدا بسته‌های لازم روی هر سه ماشین مجازی نصب شدند. دستورات اصلی نصب به شکل زیر هستند:

```
sudo apt update
sudo apt install -y \
    snmp \
    snmpd \
    build-essential \
    g++ \
    make \
    sqlite3 \
    libsqlite3-dev
```

بسته snmp ابزارهای سمت Client مانند snmpget و snmpwalk را فراهم می‌کند. بسته snmpd شامل Agent اصلی Net-SNMP است. ابزارهای g++، make و build-essential برای کامپایل برنامه Handler مورد استفاده قرار گرفتند. بسته libsqlite3-dev نیز امکان لینک کردن برنامه ++C با کتابخانه SQLite را فراهم می‌کند.

نسخه‌های اصلی ابزارها در محیط آزمایش به صورت زیر بودند:

```
Ubuntu 22.04.5 LTS
Net-SNMP 5.9.1
g++ 11.4.0
SQLite 3.37.2
```

تنظیمات سه نود

برای جلوگیری از قرار دادن مقادیر وابسته به محیط در داخل کد، تنظیمات هر نود در فایل جداگانه‌ای ذخیره شدند. فایل‌های زیر به ترتیب مربوط به Master، Slave1 و Slave2 هستند:

```
config/master.env
config/slave1.env
config/slave2.env
```

در این فایل‌ها نام نود، مسیر پایگاه داده، آدرس Agent، پورت SNMP، Community، شبکه مجاز و OID پایه مشخص شده است.

مسیر پایگاه‌های داده در سه نود به صورت زیر است:

```
Master:
/home/amir/master/master.db
```

Slave1:

/home/amir/slave1/slave1.db

Slave2:

/home/amir/slave2/slave2.db

آدرس های شبکه نودها نیز به صورت زیر هستند:

Master: 192.168.122.22

Slave1: 192.168.122.18

Slave2: 192.168.122.190

فایل config/nodes.env برای اسکریپت های توزیع شده استفاده می شود و مشخص می کند درخواست های SNMP باید به چه نودهایی و روی چه پورتهای ارسال شوند. در اجرای اسکریپت ها روی Master، Agent محلی از طریق 127.0.0.1:1161 و دو Slave از طریق IP های شبکه خوانده می شوند.

```
===== MASTER CONFIGURATION =====
NODE_NAME=master
DATABASE_PATH=/home/amir/master/master.db
SNMP_LISTEN_ADDRESS=udp:1161
SNMP_COMMUNITY=hotelMonitor
SNMP_ALLOWED_NETWORK=192.168.122.0/24
SNMP_BASE_OID=.1.3.6.1.4.1.8072.9999.4
SNMP_SYS_LOCATION=Hotel_Floor_1_Master
SNMP_SYS_CONTACT=embedded-lab

===== SLAVE 1 CONFIGURATION =====
NODE_NAME=slave1
DATABASE_PATH=/home/amir/slave1/slave1.db
SNMP_LISTEN_ADDRESS=udp:1161
SNMP_COMMUNITY=hotelMonitor
SNMP_ALLOWED_NETWORK=192.168.122.0/24
SNMP_BASE_OID=.1.3.6.1.4.1.8072.9999.4
SNMP_SYS_LOCATION=Hotel_Floor_2_Slave1
SNMP_SYS_CONTACT=embedded-lab
```

```
===== SLAVE 2 CONFIGURATION =====
NODE_NAME=slave2
DATABASE_PATH=/home/amir/slave2/slave2.db
SNMP_LISTEN_ADDRESS=udp:1161
SNMP_COMMUNITY=hotelMonitor
SNMP_ALLOWED_NETWORK=192.168.122.0/24
SNMP_BASE_OID=.1.3.6.1.4.1.8072.9999.4
SNMP_SYS_LOCATION=Hotel_Floor_3_Slave2
SNMP_SYS_CONTACT=embedded-lab

===== DISTRIBUTED NODES =====
SNMP_VERSION=2c
SNMP_COMMUNITY=hotelMonitor
SNMP_BASE_OID=.1.3.6.1.4.1.8072.9999.4
SNMP_TIMEOUT=2
SNMP_RETRIES=1

# Run read_all_sensors.sh on the Master VM. The Master agent is local and
# the two Slave agents are reached through their VM addresses.
NODES=(
    "master|127.0.0.1|1161"
    "slave1|192.168.122.18|1161"
    "slave2|192.168.122.190|1161"
)
amir-hossein-motiei@amir-hossein-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/04$
```

پیکربندی snmpd

قالب اصلی پیکربندی snmpd در فایل زیر قرار دارد:

```
snmp/snmpd.conf.template
```

این فایل شامل متغیرهایی است که هنگام اجرا با اطلاعات فایل تنظیمات نود جایگزین می شوند. بخش های اصلی قالب به شکل زیر هستند:

```
agentaddress @SNMP_LISTEN_ADDRESS@
```

```
rocommunity @SNMP_COMMUNITY@ 127.0.0.1 @SNMP_BASE_OID@
```

```
rocommunity @SNMP_COMMUNITY@ @SNMP_ALLOWED_NETWORK@ @SNMP_BASE_OID@
```

```
sysLocation @SNMP_SYS_LOCATION@
```

```
sysContact @SNMP_SYS_CONTACT@
```

```
sysName @NODE_NAME@
```

```
pass @SNMP_BASE_OID@ @HANDLER_PATH@ \
```

```
--db @DATABASE_PATH@ \
```

```
--node @NODE_NAME@ \
```

```
--base-oid @SNMP_BASE_OID@
```

دستور `agentaddress` مشخص می کند Agent روی چه آدرس و پورتنی گوش دهد. در این پروژه مقدار نهایی آن برابر `udp:1161` است. این مقدار باعث می شود `snmpd` روی پورت UDP شماره ۱۱۶۱ و روی رابط های شبکه سیستم فعال باشد.

دستور `rocommunity` یک `Community` فقط خواندنی تعریف می کند `Community`. مورد استفاده در پروژه `hotelMonitor` است. یک خط دسترسی محلی از طریق `127.0.0.1` و خط دیگر دسترسی از شبکه `192.168.122.0/24` را فراهم می کند. دسترسی فقط به شاخه اختصاصی زیر محدود شده است:

```
.1.3.6.1.4.1.8072.9999.4
```

در نتیجه Client نمی تواند با این `Community` سایر شاخه های مدیریتی سیستم را بخواند و امکان نوشتن نیز ندارد.

مهم ترین بخش فایل، دستور `pass` است. این دستور تمام درخواست هایی را که به `OID` پایه سنسورها مربوط هستند به برنامه `sensor_snmp_pass` انتقال می دهد. علاوه بر `OID` درخواستی، مسیر پایگاه داده، نام نود و `OID` پایه نیز به برنامه منتقل می شوند.

```

===== SNMPD CONFIGURATION TEMPLATE =====
# This file is converted to a node-specific snmpd.conf by
# scripts/generate_snmpd_config.sh.

agentaddress @SNMP_LISTEN_ADDRESS@

# Restrict the community to the custom hotel sensor subtree.
rocommunity @SNMP_COMMUNITY@ 127.0.0.1 @SNMP_BASE_OID@
rocommunity @SNMP_COMMUNITY@ @SNMP_ALLOWED_NETWORK@ @SNMP_BASE_OID@

sysLocation @SNMP_SYS_LOCATION@
sysContact @SNMP_SYS_CONTACT@
sysName @NODE_NAME@

dontLogTCPWrappersConnects yes

# Net-SNMP appends -g/-n/-s and the requested OID to this command.
pass @SNMP_BASE_OID@ @HANDLER_PATH@ --db @DATABASE_PATH@ --node @NODE_NAME@ --base-oid @SNMP_BASE_OID@

===== PASS HANDLER SOURCE REFERENCE =====
scripts/generate_snmpd_config.sh:11: echo "Error: SNMP pass handler is not built: $HANDLER_PATH" >&2
scripts/generate_snmpd_config.sh:16:if [[ ! -f "$DATABASE_PATH" ]]; then
scripts/generate_snmpd_config.sh:17: echo "Error: SQLite database not found: $DATABASE_PATH" >&2
scripts/generate_snmpd_config.sh:27:content="{content//@SNMP_BASE_OID@/$SNMP_BASE_OID}"
scripts/generate_snmpd_config.sh:32:content="{content//@DATABASE_PATH@/$DATABASE_PATH}"
scripts/start_snmpd.sh:59:echo "Database: $DATABASE_PATH"
scripts/start_snmpd.sh:61:echo "Base OID: $SNMP_BASE_OID"
snmp/snmpd.conf.template:7:rocommunity @SNMP_COMMUNITY@ 127.0.0.1 @SNMP_BASE_OID@
snmp/snmpd.conf.template:8:rocommunity @SNMP_COMMUNITY@ @SNMP_ALLOWED_NETWORK@ @SNMP_BASE_OID@
snmp/snmpd.conf.template:17:pass @SNMP_BASE_OID@ @HANDLER_PATH@ --db @DATABASE_PATH@ --node @NODE_NAME@ --base-oid @SNMP_B
ASE_OID@

```

تولید فایل پیکربندی اختصاصی هر نود

اسکرپت زیر وظیفه تولید فایل نهایی پیکربندی را بر عهده دارد:

```
scripts/generate_snmpd_config.sh
```

این اسکرپت ابتدا بررسی می کند که برنامه Handler کامپایل شده باشد و فایل پایگاه داده نیز وجود داشته باشد. سپس متغیرهای قالب پیکربندی را با مقادیر واقعی فایل env جایگزین می کند.

فایل های نهایی تولیدشده در مسیرهای زیر قرار می گیرند:

```
generated/master/snmpd.conf
```

```
generated/slave1/snmpd.conf
```

```
generated/slave2/snmpd.conf
```

برای نمونه، دستور pass تولیدشده برای Master به صورت زیر است:

```

pass .1.3.6.1.4.1.8072.9999.4 \
/home/amir/04/snmp/sensor_snmp_pass \
--db /home/amir/master/master.db \
--node master \
--base-oid .1.3.6.1.4.1.8072.9999.4

```

```

amir@embedded-master:~/04$ cd /home/amir/04
chmod +x scripts/generate_snmpd_config.sh
./scripts/generate_snmpd_config.sh config/master.env
Generated SNMP configuration: /home/amir/04/generated/master/snmpd.conf
amir@embedded-master:~/04$ cat generated/master/snmpd.conf
# This file is converted to a node-specific snmpd.conf by
# scripts/generate_snmpd_config.sh.

agentaddress udp:1161

# Restrict the community to the custom hotel sensor subtree.
rocommunity hotelMonitor 127.0.0.1 .1.3.6.1.4.1.8072.9999.4
rocommunity hotelMonitor 192.168.122.0/24 .1.3.6.1.4.1.8072.9999.4

sysLocation Hotel_Floor_1_Master
sysContact embedded-lab
sysName master

dontLogTCPWrappersConnects yes

# Net-SNMP appends -g/-n/-s and the requested OID to this command.
pass .1.3.6.1.4.1.8072.9999.4 /home/amir/04/snmp/sensor_snmp_pass --db /home/amir/master/master.db --n
ode master --base-oid .1.3.6.1.4.1.8072.9999.4
amir@embedded-master:~/04$

```

برنامه OID های سفارشی

برنامه اصلی Handler در فایل زیر نوشته شده است:

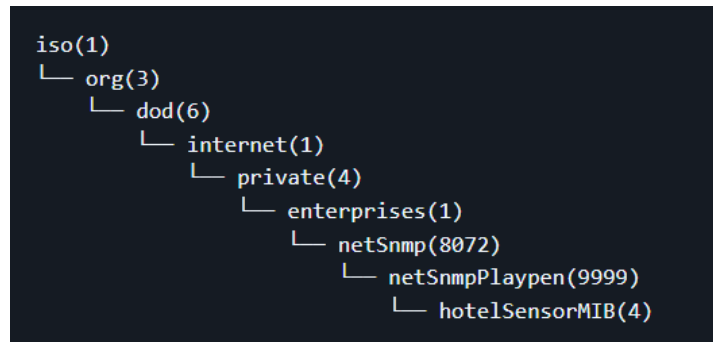
snmp/sensor_snmp_pass.cpp

این برنامه پروتکل pass مربوط به Net-SNMP را پیاده سازی می کند. هنگام دریافت یک درخواست، snmpd نوع عملیات و OID درخواست شده را به برنامه ارسال می کند. برنامه سپس OID را تحلیل کرده و مشخص می کند کدام ویژگی از کدام سنسور درخواست شده است.

در ادامه برنامه به پایگاه داده SQLite نود متصل می شود، اطلاعات سنسورها را از جدول sensors و آخرین مقدار هر سنسور را از جدول sensor_readings می خواند. نتیجه در قالب سه خط مورد انتظار Net-SNMP شامل OID، نوع داده و مقدار چاپ می شود snmpd. این خروجی را دریافت کرده و در قالب یک پاسخ استاندارد SNMP برای Client ارسال می کند.

برنامه به صورت پویا پایگاه داده را در هر درخواست می خواند. بنابراین مقادیر سنسورها داخل کد Hard-code نشده اند و پس از درج داده جدید در SQLite، مقدار جدید بدون نیاز به Restart شدن Agent قابل مشاهده است.

ساختار OID های سفارشی



OID پایه تمام اطلاعات این بخش برابر است با:

1.3.6.1.4.1.8072.9999.4

در زیر این شاخه، تعداد سنسورها و جدول اطلاعات سنسورها تعریف شده است.

Column	OID suffix	Type	Description
1	.2.1.1.<index>	INTEGER	Local SNMP table index
2	.2.1.2.<index>	STRING	Sensor ID
3	.2.1.3.<index>	STRING	Sensor name
4	.2.1.4.<index>	STRING	Sensor description: type, location, and node
5	.2.1.5.<index>	STRING	Latest stored value
6	.2.1.6.<index>	STRING	Measurement unit
7	.2.1.7.<index>	STRING	Timestamp of the latest reading
8	.2.1.8.<index>	STRING	Sensor type
9	.2.1.9.<index>	STRING	Node name

متغیر <index> شماره ردیف سنسور در جدول SNMP است و در هر نود از ۱ تا ۴ تغییر می کند. شناسه واقعی سنسور در ستون دوم جدول قرار گرفته است. برای مثال، OID زیر مقدار آخرین خوانش سنسور ردیف اول را برمی گرداند:

1.3.6.1.4.1.8072.9999.4.2.1.5.1

در Master این ردیف مربوط به سنسور 101، در Slave1 مربوط به سنسور 201 و در Slave2 مربوط به سنسور 301 است.

برای هر سنسور، علاوه بر سه فیلد الزامی نام، توضیحات و آخرین مقدار، شناسه، واحد، زمان ثبت، نوع سنسور و نام نود نیز ارائه شده است. در آزمایش انجام شده، برای سنسور اول هر نود اطلاعات کامل خوانده شد. مقدار سنسور 101 برابر 24.8 C، مقدار سنسور 201 برابر 25.3 C و مقدار سنسور 301 برابر 23.1 C بود.

```

===== MASTER SENSOR 101 / TABLE INDEX 1 =====
.1.3.6.1.4.1.8072.9999.4.2.1.2.1 = STRING: "101"
.1.3.6.1.4.1.8072.9999.4.2.1.3.1 = STRING: "Floor1_Room101_Temp"
.1.3.6.1.4.1.8072.9999.4.2.1.4.1 = STRING: "type=temperature; location=Floor1_Room101; node=master"
.1.3.6.1.4.1.8072.9999.4.2.1.5.1 = STRING: "24.8"
.1.3.6.1.4.1.8072.9999.4.2.1.6.1 = STRING: "C"
.1.3.6.1.4.1.8072.9999.4.2.1.7.1 = STRING: "2026-06-01 10:15:00"
.1.3.6.1.4.1.8072.9999.4.2.1.8.1 = STRING: "temperature"
.1.3.6.1.4.1.8072.9999.4.2.1.9.1 = STRING: "master"

===== SLAVE1 SENSOR 201 / TABLE INDEX 1 =====
.1.3.6.1.4.1.8072.9999.4.2.1.2.1 = STRING: "201"
.1.3.6.1.4.1.8072.9999.4.2.1.3.1 = STRING: "Floor2_Room201_Temp"
.1.3.6.1.4.1.8072.9999.4.2.1.4.1 = STRING: "type=temperature; location=Floor2_Room201; node=slave1"
.1.3.6.1.4.1.8072.9999.4.2.1.5.1 = STRING: "25.3"
.1.3.6.1.4.1.8072.9999.4.2.1.6.1 = STRING: "C"
.1.3.6.1.4.1.8072.9999.4.2.1.7.1 = STRING: "2026-06-01 10:15:00"
.1.3.6.1.4.1.8072.9999.4.2.1.8.1 = STRING: "temperature"
.1.3.6.1.4.1.8072.9999.4.2.1.9.1 = STRING: "slave1"

===== SLAVE2 SENSOR 301 / TABLE INDEX 1 =====
.1.3.6.1.4.1.8072.9999.4.2.1.2.1 = STRING: "301"
.1.3.6.1.4.1.8072.9999.4.2.1.3.1 = STRING: "Floor3_Room301_Temp"
.1.3.6.1.4.1.8072.9999.4.2.1.4.1 = STRING: "type=temperature; location=Floor3_Room301; node=slave2"
.1.3.6.1.4.1.8072.9999.4.2.1.5.1 = STRING: "23.1"
.1.3.6.1.4.1.8072.9999.4.2.1.6.1 = STRING: "C"
.1.3.6.1.4.1.8072.9999.4.2.1.7.1 = STRING: "2026-06-01 10:15:00"
.1.3.6.1.4.1.8072.9999.4.2.1.8.1 = STRING: "temperature"
.1.3.6.1.4.1.8072.9999.4.2.1.9.1 = STRING: "slave2"
amir@embedded-master:~/04$

```

کامپایل برنامه

برای کامپایل Handler از دو Makefile استفاده شده است. Makefile اصلی در ریشه بخش چهارم و Makefile برنامه در پوشه snmp قرار دارند:

Makefile

snmp/Makefile

دستور کامپایل برنامه به صورت زیر است:

make clean

make

برنامه با استاندارد C++17 و گزینه های بررسی هشدار کامپایل می شود:

g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic \

sensor_snmp_pass.cpp -lsqlite3 -o sensor_snmp_pass

گزینه -lsqlite3 برنامه را به کتابخانه SQLite متصل می کند. کامپایل روی هر سه نود بدون خطا و بدون هشدار انجام شد.

```

amir@embedded-master:~/04$ make clean
make -C snmp clean
make[1]: Entering directory '/home/amir/04/snmp'
rm -f sensor_snmp_pass
make[1]: Leaving directory '/home/amir/04/snmp'
rm -rf generated/* test-output/*
amir@embedded-master:~/04$ make
make -C snmp
make[1]: Entering directory '/home/amir/04/snmp'
g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic sensor_snmp_pass.cpp -lsqlite3 -o sensor_snmp_pass
make[1]: Leaving directory '/home/amir/04/snmp'
amir@embedded-master:~/04$

```

اجرای سرویس SNMP

برای ساده کردن فرایند کامپایل و اجرا، اسکریپت زیر تهیه شد:

`scripts/build_and_run.sh`

این اسکریپت ابتدا نسخه قبلی برنامه و فایل های تولید شده را پاک می کند، سپس برنامه ++C را با Makefile کامپایل کرده و در مرحله دوم Agent اختصاصی نود را اجرا می کند.

برای اجرای Master از دستور زیر استفاده می شود:

`./scripts/build_and_run.sh config/master.env`

برای Slave1 و Slave2 نیز به ترتیب دستورات زیر اجرا می شوند:

`./scripts/build_and_run.sh config/slave1.env`

`./scripts/build_and_run.sh config/slave2.env`

اسکریپت `start_snmpd.sh` فایل تنظیمات اختصاصی نود را تولید کرده و `snmpd` را در حالت Foreground داخلی، ولی به صورت یک فرایند پس زمینه مدیریت شده اجرا می کند. شناسه فرایند در فایل PID ذخیره می شود و خروجی Agent نیز در فایل Log قرار می گیرد.

برای جلوگیری از نیاز به مجوز نوشتن در `/var/lib/snmp`، یک پوشه Persistent خصوصی برای هر نود در مسیر `generated` ایجاد شده است. این تصمیم باعث شد Agent با کاربر عادی اجرا شود و نیاز به اجرای کل سرویس با دسترسی Root وجود نداشته باشد.

Agent روی آدرس `0.0.0.0:1161` گوش می دهد؛ بنابراین هم درخواست های محلی و هم درخواست های شبکه مجاز را دریافت می کند.

```
===== STOP CURRENT MASTER AGENT =====
SNMP agent stopped for master.

===== BUILD AND START MASTER SNMP AGENT =====
Step 1/2: Building the C++ SNMP pass handler...
make: Entering directory '/home/amir/04'
make -C snmp clean
make[1]: Entering directory '/home/amir/04/snmp'
rm -f sensor_snmp_pass
make[1]: Leaving directory '/home/amir/04/snmp'
rm -rf generated/* test-output/*
make -C snmp
make[1]: Entering directory '/home/amir/04/snmp'
g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic sensor_snmp_pass.cpp -lsqlite3 -o sensor_snmp_pass
make[1]: Leaving directory '/home/amir/04/snmp'
make: Leaving directory '/home/amir/04'

Step 2/2: Starting the node-specific snmpd agent...
Generated SNMP configuration: /home/amir/04/generated/master/snmpd.conf
SNMP agent started.
Node:      master
Database:  /home/amir/master/master.db
Listen:    udp:1161
Base OID:  .1.3.6.1.4.1.8072.9999.4
PID:       14855
Log:       /home/amir/04/generated/master/snmpd.log
```

```

===== MASTER AGENT VERIFICATION =====
PID: 14855
UID      PID      PPID  C  STIME TTY          TIME CMD
amir      14855      1   0 12:38 pts/0        00:00:00 snmpd -f -Lo -C -I -smux -c /home/amir/04/generate

Listening UDP port:
UNCONN 0      0          0.0.0.0:1161      0.0.0.0:*      users:({"snmpd",pid=14855,fd=6})

SNMP sensor count:
.1.3.6.1.4.1.8072.9999.4.1.0 = INTEGER: 4

SNMP agent log:
Created directory: /home/amir/04/generated/master/persistent/cert_indexes
NET-SNMP version 5.9.1
amir@embedded-master:~/04$

```

اجرای مستقل Slave1 نیز با موفقیت انجام شد. این نود به پایگاه داده /home/amir/slave1/slave1.db متصل شد و چهار سنسور با شناسه های 201 تا 204 را ارائه کرد.

```

===== STOP CURRENT SLAVE1 AGENT =====
SNMP agent stopped for slave1.

===== BUILD AND START SLAVE1 SNMP AGENT =====
Step 1/2: Building the C++ SNMP pass handler...
make: Entering directory '/home/amir/04'
make -C snmp clean
make[1]: Entering directory '/home/amir/04/snmp'
rm -f sensor_snmp_pass
make[1]: Leaving directory '/home/amir/04/snmp'
rm -rf generated/* test-output/*
make -C snmp
make[1]: Entering directory '/home/amir/04/snmp'
g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic sensor_snmp_pass.cpp -lsqlite3 -o sensor_snmp_pass
make[1]: Leaving directory '/home/amir/04/snmp'
make: Leaving directory '/home/amir/04'

Step 2/2: Starting the node-specific snmpd agent...
Generated SNMP configuration: /home/amir/04/generated/slave1/snmpd.conf
SNMP agent started.
Node:      slave1
Database:  /home/amir/slave1/slave1.db
Listen:    udp:1161
Base OID:  .1.3.6.1.4.1.8072.9999.4
PID:       14780
Log:       /home/amir/04/generated/slave1/snmpd.log

===== SLAVE1 AGENT VERIFICATION =====
Hostname: embedded-slave1
IP address: 192.168.122.18
PID: 14780
UID      PID      PPID  C  STIME TTY          TIME CMD
amir      14780      1   0 12:40 pts/0        00:00:00 snmpd -f -Lo -C -I -smux -c /home/amir/04/generate

Listening UDP port:
UNCONN 0      0          0.0.0.0:1161      0.0.0.0:*      users:({"snmpd",pid=14780,fd=6})

SNMP sensor count:
.1.3.6.1.4.1.8072.9999.4.1.0 = INTEGER: 4

SLAVE1 database sensors:
sensor_id  sensor_name                sensor_type
-----
201        Floor2_Room201_Temp        temperature
202        Floor2_Room201_Humidity    humidity
203        Floor2_Corridor_Motion     motion
204        Floor2_Meeting_CO2         co2
amir@embedded-slave1:~/04$

```

در Slave2 نیز برنامه کامپایل و Agent راه اندازی شد. این نود به پایگاه داده /home/amir/slave2/slave2.db متصل شد و سنسورهای 301 تا 304 را ارائه کرد.

```
===== STOP CURRENT SLAVE2 AGENT =====
SNMP agent stopped for slave2.

===== BUILD AND START SLAVE2 SNMP AGENT =====
Step 1/2: Building the C++ SNMP pass handler...
make: Entering directory '/home/amir/04'
make -C snmp clean
make[1]: Entering directory '/home/amir/04/snmp'
rm -f sensor_snmp_pass
make[1]: Leaving directory '/home/amir/04/snmp'
rm -rf generated/* test-output/*
make -C snmp
make[1]: Entering directory '/home/amir/04/snmp'
g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic sensor_snmp_pass.cpp -lsqlite3 -o sensor_snmp_pass
make[1]: Leaving directory '/home/amir/04/snmp'
make: Leaving directory '/home/amir/04'

Step 2/2: Starting the node-specific snmpd agent...
Generated SNMP configuration: /home/amir/04/generated/slave2/snmpd.conf
SNMP agent started.
Node:      slave2
Database:  /home/amir/slave2/slave2.db
Listen:    udp:1161
Base OID:  .1.3.6.1.4.1.8072.9999.4
PID:       3400
Log:       /home/amir/04/generated/slave2/snmpd.log

===== SLAVE2 AGENT VERIFICATION =====
Hostname: embedded-slave2
IP address: 192.168.122.190
PID: 3400
UID      PID      PPID    C  STIME TTY          TIME CMD
amir     3400         1   0  12:42 pts/0      00:00:00 snmpd -f -Lo -C -I -smux -c /home/amir/04/generate

Listening UDP port:
UNCONN 0      0              0.0.0.0:1161      0.0.0.0:*      users:((("snmpd",pid=3400,fd=6))

SNMP sensor count:
.1.3.6.1.4.1.8072.9999.4.1.0 = INTEGER: 4

SLAVE2 database sensors:
sensor_id  sensor_name              sensor_type
-----
301        Floor3_Room301_Temp      temperature
302        Floor3_Room301_Humidity  humidity
303        Floor3_Corridor_Motion   motion
304        Floor3_Storage_Smoke     smoke
amir@embedded-slave2: ~/04$
```

برای متوقف کردن Agent هر نود از اسکریپت زیر استفاده می شود:

```
./scripts/stop_snmpd.sh config/master.env
```

همین دستور با فایل تنظیمات Slave1 یا Slave2 برای توقف Agent نود مربوطه قابل استفاده است.

تست ارتباط شبکه‌ای

پیش از اجرای درخواست‌های توزیع‌شده، ارتباط IP میان Master و دو Slave با دستور ping بررسی شد. هر سه بسته ارسال شده به Slave1 و Slave2 بدون Packet Loss دریافت شدند.

نتیجه ارتباط با Slave1 برابر 0% packet loss و میانگین زمان پاسخ تقریباً 0.614 ms بود. برای Slave2 نیز Packet Loss صفر و میانگین زمان پاسخ حدود 0.937 ms ثبت شد.

پس از تأیید ارتباط IP، نود Master با استفاده از snmpget به Agent های هر دو Slave متصل شد. هر دو Agent مقدار تعداد سنسورها را برابر ۴ برگرداندند. این آزمایش نشان داد که ارتباط فقط در سطح ICMP برقرار نیست و سرویس SNMP نیز از طریق UDP پورت ۱۶۱ قابل دسترسی است.

```
===== NETWORK CONNECTIVITY TEST =====
Ping Slave1:
PING 192.168.122.18 (192.168.122.18) 56(84) bytes of data.
64 bytes from 192.168.122.18: icmp_seq=1 ttl=64 time=0.443 ms
64 bytes from 192.168.122.18: icmp_seq=2 ttl=64 time=0.671 ms
64 bytes from 192.168.122.18: icmp_seq=3 ttl=64 time=0.728 ms

--- 192.168.122.18 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2019ms
rtt min/avg/max/mdev = 0.443/0.614/0.728/0.123 ms

Ping Slave2:
PING 192.168.122.190 (192.168.122.190) 56(84) bytes of data.
64 bytes from 192.168.122.190: icmp_seq=1 ttl=64 time=1.18 ms
64 bytes from 192.168.122.190: icmp_seq=2 ttl=64 time=0.855 ms
64 bytes from 192.168.122.190: icmp_seq=3 ttl=64 time=0.778 ms

--- 192.168.122.190 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 0.778/0.937/1.180/0.174 ms

===== REMOTE SNMP CONNECTIVITY =====

Slave1 SNMP sensor count:
.1.3.6.1.4.1.8072.9999.4.1.0 = INTEGER: 4

Slave2 SNMP sensor count:
.1.3.6.1.4.1.8072.9999.4.1.0 = INTEGER: 4
amir@embedded-master:~/04$
```

خواندن اطلاعات با snmpget

برای خواندن یک OID مشخص از دستور snmpget استفاده می‌شود. قالب عمومی دستور در این پروژه به شکل زیر است:

```
snmpget \
-v2c \
-c hotelMonitor \
-On \
udp:127.0.0.1:1161 \
.1.3.6.1.4.1.8072.9999.4.1.0
```

گزینه -v2c استفاده از SNMP نسخه ۲ را مشخص می‌کند. گزینه -c Community را تعیین می‌کند. گزینه -On باعث نمایش کامل عددی OID می‌شود و آدرس udp:127.0.0.1:1161 مقصد درخواست را مشخص می‌کند.

پاسخ OID تعداد سنسورها روی هر سه نود به صورت زیر بود:

.1.3.6.1.4.1.8072.9999.4.1.0 = INTEGER: 4

```
amir@embedded-master:~/04$ snmpget \
-v2c \
-c hotelMonitor \
-On \
udp:127.0.0.1:1161 \
.1.3.6.1.4.1.8072.9999.4.1.0
.1.3.6.1.4.1.8072.9999.4.1.0 = INTEGER: 4
amir@embedded-master:~/04$
```

خواندن اطلاعات با snmpwalk

ابزار snmpwalk تمام OID های موجود در زیر یک شاخه را به ترتیب پیمایش می کند. برای خواندن کامل اطلاعات سنسورهای Master از دستور زیر استفاده شد:

```
snmpwalk \
-v2c \
-c hotelMonitor \
-On \
udp:127.0.0.1:1161 \
.1.3.6.1.4.1.8072.9999.4
```

خروجی این فرمان ابتدا تعداد سنسورها را نمایش داد و سپس تمام ستون های جدول سنسورها شامل شماره ردیف، شناسه، نام، توضیحات، آخرین مقدار، واحد، زمان ثبت، نوع سنسور و نام نود را خواند.

چهار سنسور Master عبارت بودند از:

```
101 Floor1_Room101_Temp
102 Floor1_Room101_Humidity
103 Floor1_Corridor_Motion
104 Floor1_Lobby_Temp
```

مقادیر آخر آنها نیز به ترتیب 24.8 C، 45 %، 1 bool و 23.9 C دریافت شدند.

پیام زیر در پایان خروجی مشاهده شد:

```
No more variables left in this MIB View
```

این پیام خطا نیست و تنها نشان می دهد snmpwalk به انتهای شاخه OID اختصاصی رسیده است.

خروجی کامل Master در فایل زیر ذخیره شد:

```
test-output/master_snmpwalk.txt
```

```

===== MASTER SNMPWALK TEST =====
Target: udp:127.0.0.1:1161
Base OID: .1.3.6.1.4.1.8072.9999.4

.1.3.6.1.4.1.8072.9999.4.1.0 = INTEGER: 4
.1.3.6.1.4.1.8072.9999.4.2.1.1.1 = INTEGER: 1
.1.3.6.1.4.1.8072.9999.4.2.1.1.2 = INTEGER: 2
.1.3.6.1.4.1.8072.9999.4.2.1.1.3 = INTEGER: 3
.1.3.6.1.4.1.8072.9999.4.2.1.1.4 = INTEGER: 4
.1.3.6.1.4.1.8072.9999.4.2.1.2.1 = STRING: "101"
.1.3.6.1.4.1.8072.9999.4.2.1.2.2 = STRING: "102"
.1.3.6.1.4.1.8072.9999.4.2.1.2.3 = STRING: "103"
.1.3.6.1.4.1.8072.9999.4.2.1.2.4 = STRING: "104"
.1.3.6.1.4.1.8072.9999.4.2.1.3.1 = STRING: "Floor1 Room101 Temp"
.1.3.6.1.4.1.8072.9999.4.2.1.3.2 = STRING: "Floor1 Room101 Humidity"
.1.3.6.1.4.1.8072.9999.4.2.1.3.3 = STRING: "Floor1 Corridor Motion"
.1.3.6.1.4.1.8072.9999.4.2.1.3.4 = STRING: "Floor1 Lobby Temp"
.1.3.6.1.4.1.8072.9999.4.2.1.4.1 = STRING: "type=temperature; location=Floor1 Room101; node=master"
.1.3.6.1.4.1.8072.9999.4.2.1.4.2 = STRING: "type=humidity; location=Floor1 Room101; node=master"
.1.3.6.1.4.1.8072.9999.4.2.1.4.3 = STRING: "type=motion; location=Floor1 Corridor; node=master"
.1.3.6.1.4.1.8072.9999.4.2.1.4.4 = STRING: "type=temperature; location=Floor1 Lobby; node=master"

.1.3.6.1.4.1.8072.9999.4.2.1.5.1 = STRING: "24.8"
.1.3.6.1.4.1.8072.9999.4.2.1.5.2 = STRING: "45"
.1.3.6.1.4.1.8072.9999.4.2.1.5.3 = STRING: "1"
.1.3.6.1.4.1.8072.9999.4.2.1.5.4 = STRING: "23.9"
.1.3.6.1.4.1.8072.9999.4.2.1.6.1 = STRING: "C"
.1.3.6.1.4.1.8072.9999.4.2.1.6.2 = STRING: "%"
.1.3.6.1.4.1.8072.9999.4.2.1.6.3 = STRING: "bool"
.1.3.6.1.4.1.8072.9999.4.2.1.6.4 = STRING: "C"
.1.3.6.1.4.1.8072.9999.4.2.1.7.1 = STRING: "2026-06-01 10:15:00"
.1.3.6.1.4.1.8072.9999.4.2.1.7.2 = STRING: "2026-06-01 10:15:00"
.1.3.6.1.4.1.8072.9999.4.2.1.7.3 = STRING: "2026-06-01 10:16:00"
.1.3.6.1.4.1.8072.9999.4.2.1.7.4 = STRING: "2026-06-01 10:20:00"
.1.3.6.1.4.1.8072.9999.4.2.1.8.1 = STRING: "temperature"
.1.3.6.1.4.1.8072.9999.4.2.1.8.2 = STRING: "humidity"
.1.3.6.1.4.1.8072.9999.4.2.1.8.3 = STRING: "motion"
.1.3.6.1.4.1.8072.9999.4.2.1.8.4 = STRING: "temperature"
.1.3.6.1.4.1.8072.9999.4.2.1.9.1 = STRING: "master"
.1.3.6.1.4.1.8072.9999.4.2.1.9.2 = STRING: "master"
.1.3.6.1.4.1.8072.9999.4.2.1.9.3 = STRING: "master"
.1.3.6.1.4.1.8072.9999.4.2.1.9.4 = STRING: "master"
.1.3.6.1.4.1.8072.9999.4.2.1.9.4 = No more variables left in this MIB View (It is past the end of the MIB tree)
amir@embedded-master:~/04$ echo
echo "===== SAVED WALK OUTPUT ====="
wc -l test-output/master_snmpwalk.txt
ls -lh test-output/master_snmpwalk.txt

===== SAVED WALK OUTPUT =====
38 test-output/master_snmpwalk.txt
-rw-rw-r-- 1 amir amir 2.3K Jul 21 12:45 test-output/master_snmpwalk.txt
amir@embedded-master:~/04$

```

اسکرپت تست خودکار SNMP

برای بررسی خودکار هر سه نود، اسکرپت زیر نوشته شده است:

scripts/test_snmp.sh

اسکرپت با خواندن فایل config/nodes.env به ترتیب Master، Slave1 و Slave2 را بررسی می کند. در هر نود ابتدا تعداد سنسورها خوانده می شود و سپس وجود نام، توضیحات و آخرین مقدار تمام سنسورها کنترل می شود.

فرمان اجرای تست به شکل زیر است:

./scripts/test_snmp.sh config/nodes.env

خروجی تست برای هر سه نود برابر PASS بود:

Testing master at 127.0.0.1:1161 ...

PASS: 4 sensors; name/description/latest-value are available for all.

Testing slave1 at 192.168.122.18:1161 ...

PASS: 4 sensors; name/description/latest-value are available for all.

Testing slave2 at 192.168.122.190:1161 ...

PASS: 4 sensors; name/description/latest-value are available for all.

All SNMP node tests passed.

این نتیجه نشان می دهد سه فیلد الزامی استاد برای هر ۱۲ سنسور قابل دسترسی هستند.

```
===== AUTOMATED DISTRIBUTED SNMP TEST =====
Test configuration: config/nodes.env

Testing master at 127.0.0.1:1161 ...
  PASS: 4 sensors; name/description/latest-value are available for all.
Testing slave1 at 192.168.122.18:1161 ...
  PASS: 4 sensors; name/description/latest-value are available for all.
Testing slave2 at 192.168.122.190:1161 ...
  PASS: 4 sensors; name/description/latest-value are available for all.
All SNMP node tests passed.

===== SAVED TEST RESULT =====
Testing master at 127.0.0.1:1161 ...
  PASS: 4 sensors; name/description/latest-value are available for all.
Testing slave1 at 192.168.122.18:1161 ...
  PASS: 4 sensors; name/description/latest-value are available for all.
Testing slave2 at 192.168.122.190:1161 ...
  PASS: 4 sensors; name/description/latest-value are available for all.
All SNMP node tests passed.
-rw-rw-r-- 1 amir amir 366 Jul 21 12:47 test-output/automated_snmp_test.txt
amir@embedded-master:~/04$
```

خواندن خلاصه تمام سنسورها

برای بررسی یکپارچه کل سیستم، اطلاعات اصلی تمام سنسورها از طریق Master جمع آوری شدند. در مجموع چهار سنسور از Master، چهار سنسور از Slave1 و چهار سنسور از Slave2 خوانده شدند.

سنسورهای Master شامل سنسور دما و رطوبت اتاق ۱۰۱، سنسور حرکت راهروی طبقه اول و سنسور دمای لابی بودند. سنسورهای Slave1 شامل سنسور دما و رطوبت اتاق ۲۰۱، سنسور حرکت راهروی طبقه دوم و سنسور CO2 اتاق جلسه بودند. در Slave2 نیز سنسور دما و رطوبت اتاق ۳۰۱، سنسور حرکت راهروی طبقه سوم و سنسور دود انبار قرار داشتند.

مقادیر خوانده شده به صورت زیر بودند:

```

===== ALL SENSORS THROUGH SNMP =====
Node: master (127.0.0.1:1161)
Sensor ID | Sensor Name | Latest Value
-----+-----+-----
101 | Floor1_Room101_Temp | 24.8 C
102 | Floor1_Room101_Humidity | 45 %
103 | Floor1_Corridor_Motion | 1 bool
104 | Floor1_Lobby_Temp | 23.9 C

Node: slave1 (192.168.122.18:1161)
Sensor ID | Sensor Name | Latest Value
-----+-----+-----
201 | Floor2_Room201_Temp | 25.3 C
202 | Floor2_Room201_Humidity | 50 %
203 | Floor2_Corridor_Motion | 0 bool
204 | Floor2_Meeting_CO2 | 735 ppm

Node: slave2 (192.168.122.190:1161)
Sensor ID | Sensor Name | Latest Value
-----+-----+-----
301 | Floor3_Room301_Temp | 23.1 C
302 | Floor3_Room301_Humidity | 41 %
303 | Floor3_Corridor_Motion | 0 bool
304 | Floor3_Storage_Smoke | 0 bool

===== SAVED SUMMARY =====
12
-rw-rw-r-- 1 amir amir 1.1K Jul 21 12:48 test-output/all_sensors_summary.txt
amir@embedded-master:~/04$

```

اسکرپت خواندن همه سنسورها

یکی از الزامات اصلی پروژه، ارائه اسکرپتی برای خواندن تمام سنسورها از طریق SNMP بود. این اسکرپت در مسیر زیر قرار دارد:

`scripts/read_all_sensors.sh`

فرمان اجرای آن به صورت زیر است:

`./scripts/read_all_sensors.sh config/nodes.env`

اسکرپت ابتدا فایل تنظیمات نودها را خوانده و سپس برای هر نود یک `snmpwalk` روی `OID` پایه اجرا می کند. خروجی هر نود علاوه بر نمایش در ترمینال، در فایل جداگانه ای ذخیره می شود:

`test-output/master_snmpwalk.txt`

`test-output/slave1_snmpwalk.txt`

`test-output/slave2_snmpwalk.txt`

همچنین خروجی ترکیبی تمام نودها در فایل زیر قرار گرفت:

`test-output/final_snmp_test.txt`

در تست انجام شده، کد خروجی اسکرپت برابر صفر بود. در فایل `Master` چهار شناسه، در فایل `Slave1` چهار شناسه و در فایل `Slave2` نیز چهار شناسه یافت شد. تعداد کل سنسورها ۱۲ بود و نتیجه نهایی اسکرپت به صورت موفق ثبت شد.

```

===== RUN READ-ALL-SENSORS SCRIPT =====
Command: ./scripts/read_all_sensors.sh config/nodes.env

Script exit status: 0

===== GENERATED NODE OUTPUT FILES =====
-rw-rw-r-- 1 amir amir 7.2K Jul 21 12:52 test-output/final_snmp_test.txt
-rw-rw-r-- 1 amir amir 2.3K Jul 21 12:52 test-output/master_snmpwalk.txt
-rw-rw-r-- 1 amir amir 2.3K Jul 21 12:52 test-output/slave1_snmpwalk.txt
-rw-rw-r-- 1 amir amir 2.3K Jul 21 12:52 test-output/slave2_snmpwalk.txt

===== SENSOR COUNT PER NODE =====
master sensor IDs found: 4
slave1 sensor IDs found: 4
slave2 sensor IDs found: 4

```

```

===== ALL SENSOR IDS =====
.1.3.6.1.4.1.8072.9999.4.2.1.2.1 = STRING: "101"
.1.3.6.1.4.1.8072.9999.4.2.1.2.2 = STRING: "102"
.1.3.6.1.4.1.8072.9999.4.2.1.2.3 = STRING: "103"
.1.3.6.1.4.1.8072.9999.4.2.1.2.4 = STRING: "104"
.1.3.6.1.4.1.8072.9999.4.2.1.2.1 = STRING: "201"
.1.3.6.1.4.1.8072.9999.4.2.1.2.2 = STRING: "202"
.1.3.6.1.4.1.8072.9999.4.2.1.2.3 = STRING: "203"
.1.3.6.1.4.1.8072.9999.4.2.1.2.4 = STRING: "204"
.1.3.6.1.4.1.8072.9999.4.2.1.2.1 = STRING: "301"
.1.3.6.1.4.1.8072.9999.4.2.1.2.2 = STRING: "302"
.1.3.6.1.4.1.8072.9999.4.2.1.2.3 = STRING: "303"
.1.3.6.1.4.1.8072.9999.4.2.1.2.4 = STRING: "304"

===== FINAL VALIDATION =====
Total sensor IDs found: 12
PASS: read_all_sensors.sh read all 12 sensors from all three nodes.
amir@embedded-master:~/04$

```

تست مستقیم Handler قابلیت pass

برای آنکه عملکرد برنامه ++C مستقل از snmpd نیز بررسی شود، اسکریپت زیر تهیه شد:

scripts/test_pass_handler.sh

این اسکریپت مستقیماً ورودی‌هایی مشابه ورودی Net-SNMP را به برنامه Handler ارسال می‌کند و پاسخ‌های آن را بررسی می‌کند. در ابتدا خط واقعی pass از فایل تولیدشده Master نمایش داده شد. سپس تست مستقیم Handler انجام شد و نتیجه زیر به دست آمد:

PASS: direct Net-SNMP pass protocol test succeeded.

Node: master

Sensors: 4

Required fields: name, description, latest value

کد خروجی برنامه برابر صفر بود. بنابراین مشخص شد برنامه sensor_snmp_pass پروتکل مورد انتظار قابلیت pass را به درستی پیاده‌سازی کرده است.

```

===== GENERATED PASS DIRECTIVE =====
pass .1.3.6.1.4.1.8072.9999.4 /home/amir/04/snmp/sensor_snmp_pass --db /home/amir/master/master.db --n
ode master --base-oid .1.3.6.1.4.1.8072.9999.4

===== DIRECT NET-SNMP PASS HANDLER TEST =====
PASS: direct Net-SNMP pass protocol test succeeded.
Node: master
Sensors: 4
Required fields: name, description, latest value

===== TEST RESULT =====
Exit status: 0
-rw-rw-r-- 1 amir amir 125 Jul 21 12:53 test-output/direct_pass_handler_test.txt
PASS: The custom C++ handler implements the Net-SNMP pass protocol correctly.
amir@embedded-master:~/04$

```

مسیر دریافت داده از پایگاه داده تا خروجی SNMP

مسیر کامل دریافت اطلاعات در این پروژه به شکل زیر است:

snmpwalk یا snmpget

↓

ارسال درخواست SNMP v2c روی UDP پورت ۱۶۱

↓

دریافت درخواست توسط snmpd

↓

تشخیص قرار داشتن OID در شاخه اختصاصی سنسورها

↓

اجرای دستور pass

↓

اجرای برنامه sensor_snmp_pass

↓

تحلیل OID و تشخیص سنسور و فیلد درخواستی

↓

خواندن اطلاعات از SQLite

↓

تولید خروجی در قالب Net-SNMP pass

↓

دریافت خروجی توسط snmpd

↓

ارسال پاسخ SNMP به Client

در زمان خواندن نام و توضیحات سنسور، اطلاعات از جدول sensors دریافت می شوند. هنگام خواندن آخرین مقدار و زمان ثبت، برنامه جدول sensor_readings را براساس شناسه سنسور جست و جو کرده و جدیدترین رکورد را انتخاب می کند.

این معماری باعث می شود snmpd نیازی به شناخت ساختار داخلی SQLite نداشته باشد. همچنین برنامه ++C تنها مسئول تبدیل میان OID و داده های پایگاه داده است.

تست پویایی داده

برای اثبات اینکه مقادیر SNMP از پایگاه داده خوانده می شوند و داخل برنامه ثابت نیستند، یک تست پویا روی سنسور 101 انجام شد.

در ابتدای آزمایش، آخرین مقدار سنسور در SQLite برابر 24.8 و زمان آن برابر 2026-06-01 10:15:00 بود. همین مقادیر از طریق OID های مقدار و زمان در SNMP نیز دریافت شدند.

سپس یک رکورد آزمایشی با مقدار 99.9 و زمان 2099-12-31 23:59:59 در جدول sensor_readings درج شد. بلافاصله و بدون Restart کردن snmpd، OID های مربوطه دوباره خوانده شدند. این بار SNMP مقدار 99.9 و زمان جدید را برگرداند.

شناسه فرایند Agent در تمام مدت برابر 14855 باقی ماند. در نتیجه مشخص شد تغییر پاسخ به دلیل Restart شدن Agent یا بارگذاری مجدد تنظیمات نبوده است و Handler در هر درخواست جدید اطلاعات را مستقیماً از SQLite خوانده است.

پس از پایان تست، رکورد آزمایشی حذف شد. مقدار SNMP دوباره به 24.8 و زمان اولیه بازگشت. این آزمایش مهم ترین مدرک برای اثبات جریان پویای داده از SQLite تا SNMP محسوب می شود.

```
===== DYNAMIC SQLITE TO SNMP TEST =====
SNMP agent PID before test: 14855

===== 1. ORIGINAL DATABASE VALUE =====
sensor_id value recorded_at
-----
101      24.8  2026-06-01 10:15:00

Original value received through SNMP:
1.3.6.1.4.1.8072.9999.4.2.1.5.1 = STRING: "24.8"
1.3.6.1.4.1.8072.9999.4.2.1.7.1 = STRING: "2026-06-01 10:15:00"

===== 2. INSERT NEW SQLITE READING =====
sensor_id value recorded_at
-----
101      99.9  2099-12-31 23:59:59

New value received through SNMP without restarting snmpd:
1.3.6.1.4.1.8072.9999.4.2.1.5.1 = STRING: "99.9"
1.3.6.1.4.1.8072.9999.4.2.1.7.1 = STRING: "2099-12-31 23:59:59"

SNMP agent PID during test: 14855

===== 3. DELETE TEST READING =====
Restored database value:
sensor_id value recorded_at
-----
101      24.8  2026-06-01 10:15:00

Restored value received through SNMP:
1.3.6.1.4.1.8072.9999.4.2.1.5.1 = STRING: "24.8"
1.3.6.1.4.1.8072.9999.4.2.1.7.1 = STRING: "2026-06-01 10:15:00"

===== PID VERIFICATION =====
PID before test: 14855
PID after test: 14855
PASS: SNMP data changed without restarting the agent.
amir@embedded-master:~/04$
```

کنترل دسترسی و فقط خواندنی بودن Agent

در پیکربندی Agent از `rocommunity` استفاده شده است. عبارت `ro` نشان دهنده `Read Only` بودن `Community` است. برای بررسی عملی کنترل دسترسی، سه آزمایش انجام شد. در آزمایش نخست، درخواست با `Community` اشتباه `wrongCommunity` ارسال شد. هیچ پاسخی به این درخواست نداد و `Client` با پیام `Timeout` و کد خروجی ۱ متوقف شد. در آزمایش دوم، همان `OID` با `Community` صحیح `hotelMonitor` خوانده شد و تعداد سنسورها برابر ۴ دریافت شد. کد خروجی این عملیات صفر بود. در آزمایش سوم، تلاش شد مقدار سنسور با دستور `snmpset` تغییر داده شود. Agent درخواست را با خطای `noAccess` رد کرد و کد خروجی عملیات برابر ۲ بود. این نتایج نشان دادند فقط `Client` هایی که `Community` صحیح را در اختیار دارند امکان خواندن شاخه اختصاصی را دارند و حتی با `Community` صحیح نیز عملیات نوشتن مجاز نیست.

```
===== SNMP ACCESS CONTROL TEST =====
===== 1. WRONG COMMUNITY =====
Expected result: timeout or no response
Timeout: No Response from udp:127.0.0.1:1161.
Wrong-community exit status: 1

===== 2. VALID READ-ONLY COMMUNITY =====
Expected result: sensor count is readable
.1.3.6.1.4.1.8072.9999.4.1.0 = INTEGER: 4
Valid-read exit status: 0

===== 3. SNMP SET ATTEMPT =====
Expected result: write operation is rejected
Error in packet.
Reason: noAccess
Failed object: .1.3.6.1.4.1.8072.9999.4.2.1.5.1
SNMP-SET exit status: 2

===== FINAL SECURITY VALIDATION =====
PASS: Wrong community was rejected.
PASS: Valid community retained read access.
PASS: SNMP write operation was rejected.
PASS: The custom SNMP subtree is read-only.

-rw-rw-r-- 1 amir amir 777 Jul 21 12:54 test-output/snmp_access_control_test.txt
amir@embedded-master:~/04$
```

ارزیابی نهایی سیستم

در تست نهایی، وضعیت هر سه Agent به صورت همزمان از نود `Master` بررسی شد. `Master` روی آدرس محلی `127.0.0.1:1161`، `Slave1` روی آدرس `192.168.122.18:1161` و `Slave2` روی آدرس `192.168.122.190:1161` در دسترس بودند. هر سه نود تعداد سنسورها را برابر ۴ برگرداندند.

اسکرپت خودکار نیز دوباره اجرا شد و برای هر سه نود نتیجه `PASS` ثبت شد. بررسی پایگاه داده نشان داد رکورد آزمایشی مربوط به تست مقدار `99.9` به طور کامل حذف شده است و مقدار نهایی سنسور `101` همان مقدار اصلی `24.8` است.

در نتیجه نهایی، هر سه Agent عملیاتی بودند، تمام ۱۲ سنسور از طریق `SNMP` قابل دسترسی بودند و همه تست های بخش چهارم با موفقیت پایان یافتند.

```

===== FINAL SECTION 04 VALIDATION =====
Validation time: Tue Jul 21 12:55:16 PM UTC 2026
Validation host: embedded-master

===== 1. SNMP AGENT STATUS =====
Node      | Address           | Sensor Count
-----+-----+-----
master    | 127.0.0.1:1161    | 4
slave1    | 192.168.122.18:1161 | 4
slave2    | 192.168.122.190:1161 | 4

===== 2. AUTOMATED TEST =====
Testing master at 127.0.0.1:1161 ...
  PASS: 4 sensors; name/description/latest-value are available for all.
Testing slave1 at 192.168.122.18:1161 ...
  PASS: 4 sensors; name/description/latest-value are available for all.
Testing slave2 at 192.168.122.190:1161 ...
  PASS: 4 sensors; name/description/latest-value are available for all.
All SNMP node tests passed.

===== 3. DATABASE CLEANUP VERIFICATION =====
Remaining dynamic-test records: 0

Current sensor 101 reading:
sensor_id value recorded_at
-----+-----+-----
101       24.8    2026-06-01 10:15:00

===== 4. GENERATED TEST ARTIFACTS =====
total 36K
-rw-rw-r-- 1 amir amir 1.1K Jul 21 12:48 all_sensors_summary.txt
-rw-rw-r-- 1 amir amir 366 Jul 21 12:47 automated_snmp_test.txt
-rw-rw-r-- 1 amir amir 125 Jul 21 12:53 direct_pass_handler_test.txt
-rw-rw-r-- 1 amir amir 7.2K Jul 21 12:52 final_snmp_test.txt
-rw-rw-r-- 1 amir amir 2.3K Jul 21 12:52 master_snmpwalk.txt
-rw-rw-r-- 1 amir amir 2.3K Jul 21 12:52 slave1_snmpwalk.txt
-rw-rw-r-- 1 amir amir 2.3K Jul 21 12:52 slave2_snmpwalk.txt
-rw-rw-r-- 1 amir amir 777 Jul 21 12:54 snmp_access_control_test.txt

===== FINAL RESULT =====
PASS: Temporary database test record was removed.
PASS: All three SNMP nodes are operational.
PASS: All 12 distributed sensors are accessible.
PASS: Section 04 implementation and tests are complete.
amir@embedded-master:~/04$

```

مشکلات encountered و نحوه رفع آنها

در نخستین اجرای Agent سفارشی، snmpd تلاش می‌کرد فایل‌های پایدار و Cache خود را در مسیر پیش‌فرض `/var/lib/snmp` ایجاد کند. از آنجا که Agent با کاربر عادی اجرا می‌شد، دسترسی نوشتن در این مسیر وجود نداشت و خطاهایی مربوط به Permission مشاهده می‌شد. برای حل این مشکل، برای هر نود یک پوشه Persistent خصوصی در مسیر `generated` ایجاد شد. متغیر `SNMP_PERSISTENT_DIR` پیش از اجرای snmpd به این مسیر تنظیم شد. در نتیجه Agent بدون نیاز به دسترسی Root قادر به ایجاد فایل‌های داخلی خود شد. همچنین برای جلوگیری از پیام‌های متعدد مربوط به MIB های پیش‌فرض سیستم، متغیر MIBS هنگام اجرای Agent خالی در نظر گرفته شد. این کار تأثیری بر OID های عددی سفارشی پروژه ندارد و فقط از بارگذاری MIB های غیرضروری جلوگیری می‌کند. اجرای Agent روی پورت ۱۱۶۱ نیز از تداخل با سرویس پیش‌فرض snmpd سیستم روی پورت ۱۶۱ جلوگیری کرد.

تحلیل نتایج

نتایج آزمایش‌ها نشان دادند استفاده از `snmpd` همراه با قابلیت `pass` روش مناسبی برای ارائه اطلاعات سفارشی از یک پایگاه داده است. در این معماری نیازی به تغییر کد اصلی `Net-SNMP` یا نوشتن `Module` داخلی برای `snmpd` وجود ندارد `Handler`. خارجی می‌تواند با هر زبان مناسبی نوشته شود و تنها لازم است پروتکل ورودی و خروجی `pass` را رعایت کند. ساختار `OID` تعریف شده امکان اضافه کردن سنسورهای بیشتر را نیز فراهم می‌کند. چون تعداد سنسورها از پایگاه داده خوانده می‌شود و شاخص جدول به صورت پویا تولید می‌شود، اضافه شدن داده‌های جدید نیازمند تعریف دستی `OID` جداگانه در کد نیست. تفکیک تنظیمات نودها از کد نیز باعث شده یک نسخه یکسان از برنامه روی `Master`، `Slave1` و `Slave2` اجرا شود. تفاوت رفتار نودها تنها از طریق فایل `env` و مسیر پایگاه داده تعیین می‌شود. نتیجه تست پویایی داده نیز نشان داد `SNMP` اطلاعات `Cache` شده یا ثابت ارائه نمی‌کند و آخرین رکورد واقعی پایگاه داده در هر درخواست قابل دریافت است. این ویژگی برای کاربردهای مانیتورینگ اهمیت زیادی دارد؛ زیرا `Manager` باید همواره جدیدترین مقدار ذخیره شده سنسور را مشاهده کند.

در این بخش، پروتکل `SNMP` برای مانیتورینگ سنسورهای سیستم توزیع شده پیاده‌سازی شد. روی هر سه نود یک `Agent` سفارشی مبتنی بر `snmpd` اجرا شد و اطلاعات سنسورها از طریق یک شاخه اختصاصی `OID` ارائه شدند. برای ارتباط میان `snmpd` و `SQLite` از قابلیت `pass` استفاده شد. برنامه `sensor_snmp_pass` درخواست‌های `SNMP` را تحلیل کرده، اطلاعات مورد نیاز را از پایگاه داده محلی نود خوانده و پاسخ مناسب را تولید می‌کند. برای هر سنسور نام، توضیحات و آخرین مقدار ذخیره شده قابل خواندن است. علاوه بر این، شناسه، واحد، زمان ثبت، نوع سنسور و نام نود نیز در ساختار `OID` قرار داده شده‌اند. ابزارهای `snmpget` و `snmpwalk` برای خواندن اطلاعات آزمایش شدند. اسکریپت `test_snmp.sh` صحت اطلاعات هر سه نود و اسکریپت `read_all_sensors.sh` خواندن تمام ۱۲ سنسور را تأیید کردند. برنامه روی هر سه نود با `Makefile` و اسکریپت `build_and_run.sh` بدون خطا کامپایل و اجرا شد. تست تغییر مستقیم داده در `SQLite` نیز ثابت کرد آخرین مقدار سنسور به صورت پویا و بدون `Restart` شدن `Agent` در خروجی `SNMP` قابل مشاهده است. آزمایش کنترل دسترسی نشان داد `Community` اشتباه رد می‌شود و شاخه اختصاصی فقط خواندنی است. بنابراین تمام الزامات فنی و موارد تحویلی بخش چهارم شامل پیگیری `snmpd`، `OID`های اختصاصی، قابلیت `pass`، فایل `MIB`، اسکریپت خواندن سنسورها، `Makefile`، اسکریپت کامپایل و اجرا، فایل `Markdown` و تست‌های عملی با موفقیت انجام شده‌اند.